ENTERPRISE PROJECT SPECIFICATION DOCUMENT

SECTION 1 — EXECUTIVE SUMMARY

The exponential growth of digital payments, online banking platforms, and mobile financial services has created a massive increase in transaction volume and user accessibility. Alongside this expansion, the sophistication of financial fraud has escalated, rendering traditional rule-based fraud detection systems ineffective. These systems are rigid, non-adaptive, prone to false positives, and incapable of identifying modern fraud patterns that evolve rapidly.

Moreover, existing platforms store audit logs in centralized databases, making them vulnerable to manipulation. This compromises trust, legal defensibility, and regulatory compliance.

This project proposes a **next-generation, enterprise-grade fraud detection framework** that integrates:

1. **AI-based Real-Time Fraud Detection**
   A machine learning engine that evaluates transactions instantly, identifies suspicious behavior, and adapts continuously through learning based on user behavior patterns.
2. **Custom Blockchain Framework (ProofLedger / TrustLedger)**
   A fully offline, locally simulated blockchain providing immutability, tamper-proof storage, and cryptographically verifiable audit logs.
3. **Explainable AI (XAI)**
   Transparent, human-interpretable insights explaining *why* a transaction is flagged (feature importance, risk factors, anomaly metrics).
4. **Interactive Real-Time Monitoring Dashboard**
   A multi-role interface enabling administrators, analysts, and auditors to track transactions, alerts, trends, and blockchain verification.

This integrated system addresses both critical gaps in financial fraud detection:
(i) High-accuracy real-time detection, and
(ii) Immutable evidence integrity.

The project runs fully offline, supports cross-platform deployment, and is engineered to be implemented by a single intern within 7 days following the VIBE rapid-development methodology.

SECTION 2 — PROJECT BACKGROUND & CONTEXT

Global financial ecosystems—UPI platforms, card networks, NEFT/RTGS systems, and mobile wallets—have transformed consumer behavior. Digital transactions now occur at unprecedented scale, exposing institutions to sophisticated fraud vectors such as:

- Identity theft
- Synthetic accounts
- Transaction laundering
- Card-not-present fraud
- Phishing-driven credential compromise

As highlighted in the uploaded documents, financial institutions lose 3–5% of annual revenue to fraud, costing billions worldwide annually

78_Sem-1_report

. Existing systems rely on rigid rules that cannot identify new fraud strategies and generate excessive false positives. Fraudsters adapt quickly, exploiting static thresholds and mimicking legitimate behavior.

Meanwhile, centralized storage of fraud logs creates serious challenges:

- Data tampering by insiders
- Corrupted evidence integrity
- Lack of transparent audit trails
- Legal disputes due to unverifiable logs

Blockchain solves the **trust and integrity** problem, while AI solves the **real-time detection and adaptability** challenge.

Combining AI + XAI + Blockchain creates a complete, future-ready fraud prevention architecture—exactly aligned with industry requirements, academic expectations, and FinTech regulatory frameworks.

This project reflects modern enterprise architecture thinking, integrating data engineering, AI, security engineering, cryptographic assurance, visualization, and offline deployment constraints.

SECTION 3 — TECHNICAL PROBLEM STATEMENT

Conventional fraud detection frameworks suffer from two fundamental weaknesses:

1. Detection Gap

- Systems depend heavily on manually defined rules.
- Cannot detect complex, evolving fraud patterns.

- Very high false positives (legitimate transactions wrongly flagged).
- Very high false negatives (missed fraud cases).
- Poor adaptability to new fraud behaviors.
  (As extensively documented in your literature review and problem analysis.)

  78_Sem-1_report

2. Trust & Auditability Gap

Even when detection occurs, centralized storage means:

- Logs can be altered or deleted.
- Evidence cannot be cryptographically verified.
- Regulators lack trust in institution-maintained systems.
- Legal defensibility is weak due to lack of immutability.

  KRIXION_TECH_Final_Problem_Stat…

Core Problem Definition

Banks and financial platforms urgently need a system that:

"Detects fraud in real time with high accuracy AND stores detection logs in an immutable, tamper-proof ledger to ensure complete trust, transparency, and verification."

Why AI + Blockchain is Needed

- AI → Detects anomalies with speed + accuracy.
- Blockchain → Ensures fraud evidence cannot be modified.
- Explainable AI → Provides human-readable justification for decisions.
- Offline deployment → Ensures local control, data sovereignty, and academic feasibility.

This combined approach directly addresses all detection and trust deficiencies found in existing systems.

SECTION 4 — PROJECT OBJECTIVES

Derived strictly from your uploaded Detailed Objectives

DETAILED PROJECT OBJECTIVES

and Final Objectives

KRIXION_TECH_Final_Objectives

.

4.1 GENERAL OBJECTIVE

To design and implement a **real-time, AI-powered fraud detection system** integrated with a **custom blockchain simulation (ProofLedger/TrustLedger)** and **Explainable AI**, delivering secure, transparent, accurate, and tamper-proof fraud prevention suitable for FinTech environments.

4.2 SPECIFIC OBJECTIVES

1. **AI Fraud Detection Engine**
    - Develop real-time fraud scoring using ML classification and anomaly detection.
    - Build a complete data preprocessing + feature engineering pipeline.
    - Achieve >95% accuracy with minimal false positives.
    - Implement adaptive learning mechanisms.
2. **Custom Blockchain Simulation (ProofLedger/TrustLedger)**
    - Design a local blockchain with SHA-256 hashing.
    - Implement digital signatures and block linking.
    - Provide chain integrity verification.
    - Ensure 100% offline operability.
3. **Real-Time Transaction Monitoring**
    - Instant scoring of incoming transactions.
    - Configurable risk thresholds and rule layering.
    - Dashboard auto-refresh without page reload.
    - Automated alerting for suspicious activity.
4. **Explainable AI (XAI) Integration**
    - Integrate SHAP/feature importance explanations.
    - Provide human-friendly “why this was flagged” insights.
    - Include confidence scoring and interpretability reports.
5. **Interactive Dashboard**
    - Implement monitoring, audit, analysis, and admin views.
    - Support multi-role access (Admin, Analyst, Auditor).
    - Include advanced search, filters, and visualization layers.
    - Ensure fully offline, responsive design.
6. **Visual Analytics & Reporting**
    - Generate fraud trends, heatmaps, time-series analysis.
    - Create compliance-ready export (CSV/PDF).
    - Include model performance visualization.
7. **Security & Integrity**
    - Implement input validation, encryption, and tamper detection.

- Ensure robust audit trails for regulators.
- Support blockchain-based verification of evidence.

4.3 SCOPE

In-Scope (Based on your uploaded documents)

- AI model development
- XAI integration
- Custom blockchain simulation
- Offline dashboard
- Real-time monitoring
- Local database (SQLite)
- Exportable reports
- Academic-level prototype with enterprise architecture detailing

Out-of-Scope

- Integration with real banking APIs
- Use of public blockchains (Ethereum/Hyperledger)
- Cloud-hosted ML model training
- Multi-node distributed network

SECTION 5 — FUNCTIONAL REQUIREMENTS

All functional requirements derived from uploaded objectives, problem statements, and system architecture diagrams.

5.1 Transaction Processing

- Accept input transactions (CSV, simulated live stream).
- Validate fields (amount, device, timestamp, user ID).
- Normalize and encode data.

5.2 AI Fraud Detection Engine

- Real-time scoring of each transaction.
- Probability output (0–1).

- Fraud classification based on threshold θ.
- Support for ML models: Logistic Regression, Random Forest, or anomaly detection algorithms.
- Ability to retrain with new data.

5.3 Explainable AI

- Generate feature attribution values.
- Provide SHAP-based plots/importance metrics.
- Provide textual explanations:
  "Flagged because amount is 5x above normal + new device."

5.4 Blockchain (ProofLedger/TrustLedger)

- Create blocks containing:
  - transaction data
  - AI score
  - explanation metadata
  - timestamp
  - previous block hash
- Write immutable ledger entries.
- Verify chain integrity.

custome blockchain

5.5 Dashboard & User Roles

- Real-time monitoring dashboard.
- Fraud alerts panel.
- Blockchain viewer.
- Model performance charts.
- Admin, Analyst, Auditor access control.

5.6 Reporting

- Generate PDF/CSV for:
  - alerts
  - fraud cases
  - blockchain logs
  - model performance

5.7 System Management

- Configuration of thresholds.
- Model refresh & retraining control.

- Blockchain chain reset (admin only).

READY FOR NEXT BATCH

When you confirm, I will generate:

SECTION 6 to SECTION 10

(Non-Functional Requirements, Assumptions & Constraints, System Requirements, Risk Assessment, High-Level Architecture)

Reply: **Next** to proceed.

Sources

You said:

next

ChatGPT said:

Understood. Proceeding with the next batch:

SECTION 6 to SECTION 10

All content is fully enterprise-grade and strictly aligned with your uploaded files:

- 78_Sem-1_report.docx

  78_Sem-1_report

- 78_Synopsis.docx

  78_Synopsis

- Custom blockchain.docx

  custome blockchain

- Detailed Project Objectives.docx

  DETAILED PROJECT OBJECTIVES

- KRIXION TECH Final Objectives.pdf

  KRIXION_TECH_Final_Objectives

- KRIXION TECH Final Problem Statement.pdf

  KRIXION_TECH_Final_Problem_Stat…

SECTION 6 — NON-FUNCTIONAL REQUIREMENTS (NFRs)

Non-functional requirements determine performance, security, reliability, usability, maintainability, scalability, and offline operability of the system. This section defines strict enterprise-grade NFRs suitable for FinTech environments.

6.1 Performance Requirements

1. **Real-Time Scoring Latency**
   - AI scoring must occur in **<150 ms per transaction**.
   - End-to-end processing (ingestion → scoring → blockchain logging → dashboard update) must occur in **<500 ms**.
2. **Throughput**
   - System must support **100–300 transactions/second** in simulation environment.
   - Batch uploads of 100,000+ transactions must be processed without failure.
3. **Dashboard Responsiveness**
   - Dashboard must update in **near-real-time (<1 sec lag)** without page reload.

6.2 Reliability

1. **Ledger Integrity Reliability**
   - Blockchain ledger validation must have **100% accuracy** in tamper detection.
   - No block corruption allowed.
2. **Model Reliability**
   - Fraud detection engine must maintain **95%+ accuracy** across multiple test datasets.
3. **Failure Recovery**
   - System must persist state locally.

- On restart, no blockchain or model data should be lost.

6.3 Scalability

1. **Vertical Scalability**
   - System must run efficiently on laptops with 4–8 GB RAM.
   - AI model should retrain on larger datasets (up to 1M records).
2. **Modular Scalability**
   - Each module (AI, XAI, Blockchain, Dashboard) must be independently upgradeable.

6.4 Security

1. **Local Data Protection**
   - All sensitive fields (user ID, device ID, location) must be hashed/encoded.
   - No raw data exposure in logs.
2. **Blockchain Integrity**
   - SHA-256 hashing for each block.
   - Digital signatures for authenticity.
   - Append-only write mode.
3. **Role-Based Access**
   - Admin: full control
   - Analyst: monitoring & analytics
   - Auditor: read-only
     (Based on requirements:

     DETAILED PROJECT OBJECTIVES

)

6.5 Maintainability

- Code must be modular with clear separation:
  `/ai_engine`, `/xai_engine`, `/blockchain`, `/dashboard`, `/data_pipeline`.
- Must be maintainable by a single intern.
- Clear documentation and inline comments required.

6.6 Usability

- Dashboard must be intuitive and minimalistic.
- Fast navigation between Monitoring, Alerts, Analytics, Blockchain Inspector.
- Lightweight UI frameworks (NiceGUI/Flask/Streamlit preferred).

6.7 Offline Operation

- Entire system must run **100% offline**, including:
  - AI model
  - Blockchain
  - Dashboard
  - Reports
  - Data storage
- No third-party API dependencies.
  (Confirmed in multiple project objectives and problem statements.)

  custome blockchain

SECTION 7 — ASSUMPTIONS & CONSTRAINTS

This section formalizes all technical, operational, data, and environmental assumptions & constraints.

7.1 Assumptions

1. **Simulated or Local Transaction Data**
   - System uses CSV/XLSX datasets or locally generated transaction streams.
   - No connection to real banking APIs.
     (Per scope:

     78_Sem-1_report

)
2. **Offline Execution**

- User system has Python, libraries, and local database installed.
- No cloud or public blockchain nodes required.

3. **Blockchain is Simulated Locally**
   - Custom blockchain is implemented in code (ProofLedger/TrustLedger).
   - No Ethereum/Hyperledger/Ganache.
     (As confirmed:

     custome blockchain

)
4. **Single Device Usage**
   - All components run on a single machine.
5. **User Roles Available Locally**
   - Admin, Analyst, Auditor roles are enforced within local dashboard.

7.2 Constraints

1. **Hardware Constraint**
   - System must run on mid-tier laptops:
     - RAM: 4–8 GB
     - CPU: Dual Core
2. **Local Database Constraint**
   - SQLite is mandatory for offline operation.
   - No MySQL/PostgreSQL/remote DB allowed.
3. **Model Training Time**
   - Model must train within **<2 minutes** on a standard laptop.
4. **Blockchain Size Constraint**
   - Each block stores only essential fields to keep ledger light.
5. **No Third-Party API Calls**
   - Prohibited in all modules.
6. **Academic Deliverable Constraint**
   - All results must be deterministic and reproducible on evaluator device.

SECTION 8 — SYSTEM REQUIREMENTS (HARDWARE + SOFTWARE)

8.1 Hardware Requirements

Minimum

- CPU: Dual Core
- RAM: 4 GB
- Storage: 2 GB free
- OS: Windows 10 / Linux (Ubuntu) / macOS
- GPU: Not required

Recommended

- CPU: i5 or equivalent
- RAM: 8 GB
- SSD storage
- Optional GPU for faster ML training

8.2 Software Requirements

Operating Systems Supported

- Windows 10+
- Ubuntu 20.04+
- macOS Catalina+

Programming Language

- **Python 3.10+**

Mandatory Python Libraries

- pandas
- numpy
- scikit-learn
- matplotlib
- seaborn
- shap (for explainable AI)
- hashlib (blockchain hashing)
- sqlite3
- flask / streamlit / NiceGUI

(Aligned with project files describing dashboard + AI + blockchain.)

8.3 Local Database

- SQLite (mandatory)
- No external DB dependencies
- Supports offline read/write

8.4 Installation Requirements

- Python environment setup
- Local folder structure setup
- Pre-loaded sample datasets
- No internet needed after initial installation

SECTION 9 — RISK ASSESSMENT & MITIGATION

This section analyzes risks across Data, Model, Blockchain, UI/UX, Performance, and Operational factors.

9.1 Data Risks

Risk	Impact	Mitigation
Noisy or incomplete data	Reduces model accuracy	Implement preprocessing pipeline; missing value imputation
Imbalanced fraud dataset	Bias toward “no-fraud”	Use oversampling/SMOTE; adjust threshold
Dataset too small	Poor generalization	Use synthetic data generation + augmentation

---

9.2 Model Risks

Risk	Impact	Mitigation
Overfitting	Poor real-world performance	Cross-validation, regularization
Concept drift	Decline in accuracy over time	Retraining module + periodic validation
False positives	User inconvenience	Threshold tuning, XAI transparency

---

9.3 Blockchain Risks

Risk	Impact	Mitigation
Incorrect hash linking	Ledger break	Strict append-only block validation
Block tampering	Evidence invalidated	Automatic hash-chain verification
Large ledger size	Performance overhead	Store only minimal metadata

---

9.4 UI/UX Risks

Risk	Impact	Mitigation
Dashboard lag	Poor monitoring experience	Use lightweight front-end; optimize queries
Misinterpretation of alerts	Wrong decisions by analysts	Clear color coding + XAI explanations

---

9.5 Operational Risks

Risk	Impact	Mitigation
Local system crash	Data loss	Auto-backup blockchain + DB
Incorrect role access	Security breach	Enforce RBAC strictly
Misconfiguration	Wrong thresholds	Admin-only configuration panel

---

SECTION 10 — HIGH-LEVEL SYSTEM ARCHITECTURE

This section presents the top-level architecture of the integrated AI + Blockchain + XAI + Dashboard system.

(All details aligned with your architecture diagrams inside 78_Sem-1_report.docx.)

78_Sem-1_report

10.1 Architecture Layers

1. Data Layer

- Transaction dataset (CSV/XLSX)
- SQLite database
- Preprocessing pipelines

2. AI Fraud Detection Layer

- ML classifier
- Anomaly detection algorithms
- Thresholding logic
- Adaptive learning

3. Explainable AI Layer (XAI)

- SHAP computation
- Feature importance generation
- Human-readable explanations

4. Blockchain Layer (ProofLedger / TrustLedger)

- Block creation module
- Hashing module (SHA-256)
- Chain linkage validation
- Audit retrieval

5. Dashboard & Visualization Layer

- Real-time transaction feed
- Alerts panel
- Blockchain inspector
- Analytics charts
- Export module

10.2 End-to-End Flow

1. **Transaction Input**
2. → AI Fraud Scoring
3. → XAI Explanation Generation

4. → Blockchain Logging
5. → Dashboard Visualization
6. → Reporting + Audit Verification

10.3 Architectural Strengths

- Fully modular
- Offline-capable
- Transparent and auditable
- Enterprise-ready layering
- Compatible with academic requirements

READY FOR NEXT BATCH

When you reply **next**, I will deliver:

SECTION 11 to SECTION 15

(Detailed Module Architecture, Data Flow Architecture, Dataset Specification, Dataset Sources, Dataset Preprocessing Rules)

Sources

You said:

next

ChatGPT said:

Understood. Continuing with the next batch.

Below are **SECTION 11 to SECTION 15**, fully enterprise-grade, aligned strictly with your uploaded files:

- 78_Sem-1_report.docx

  78_Sem-1_report

- 78_Synopsis.docx

78_Synopsis

- Custom blockchain.docx

  custome blockchain

- Detailed Project Objectives.docx

  DETAILED PROJECT OBJECTIVES

- KRIXION TECH Final Objectives.pdf

  KRIXION_TECH_Final_Objectives

- KRIXION TECH Final Problem Statement.pdf

  KRIXION_TECH_Final_Problem_Stat…

SECTION 11 — DETAILED MODULE ARCHITECTURE

The system comprises **five enterprise-grade modules**, each independently deployable yet tightly integrated through a well-defined data pipeline.

11.1 Module 1 — Transaction Ingestion & Preprocessing

Purpose

To intake raw transactions and transform them into ML-ready feature vectors.

Subcomponents

1. **Data Loader**
   - Reads CSV/XLSX or simulated live stream.
   - Verifies mandatory fields.
2. **Validation Pipeline**
   - Checks for missing attributes (device, timestamp, amount).
   - Detects invalid timestamps or malformed entries.
3. **Feature Engineering Engine**

    - Encodes categorical variables (merchant, device type, location).
    - Computes derived indicators:
        - spend frequency
        - location deviation
        - amount deviation from user baseline
        - device change metric
4. **Normalization Layer**
    - Min-max scaling / standardization.

Outputs

Clean, structured vector ready for the AI Fraud Detection Engine.

11.2 Module 2 — AI Fraud Detection Engine

Purpose

Real-time scoring of transactions using supervised ML and anomaly detection.

Subcomponents

1. **Model Loader**
    - Loads trained ML model (.pkl).
    - Supports Logistic Regression, Random Forest, or Isolation Forest.
2. **Inference Engine**
    - Computes fraud probability (0–1).
    - Applies threshold θ to classify "Legitimate" or "Suspicious".
3. **Adaptive Learning Component**
    - Allows incremental training.
    - Handles concept drift over time.

Internal Functions

- `predict_fraud_score()`
- `update_model()`
- `validate_scores()`

Outputs

Fraud Score + Classification Label.

11.3 Module 3 — Explainable AI (XAI) Engine

Purpose

To explain *why* a transaction was flagged.

Subcomponents

1. **SHAP Value Generator**
    - Computes feature contributions per transaction.
2. **Local Explanation Builder**
    - Converts SHAP values into human-readable text:
      "Unusual amount + new device + foreign location."
3. **Global Interpretation Engine**
    - Generates overall feature importance across dataset.

Outputs

- Feature attribution vector
- Narrative explanation
- Confidence score

11.4 Module 4 — ProofLedger / TrustLedger Blockchain

Purpose

To store fraud logs immutably and ensure tamper-proof audit structures.

(As explicitly described in your custom blockchain document.)

custome blockchain

Subcomponents

1. **Block Constructor**
    - Block = {index, timestamp, transaction data, fraud score, explanation hash, previous hash}.
2. **Hashing Engine**
    - Uses SHA-256 cryptographic hashing.
3. **Chain Manager**
    - Maintains append-only ledger.
    - Stores blocks as JSON objects in SQLite/local storage.

4. **Chain Integrity Validator**
    - Verifies if entire chain is tamper-free.
    - Used by auditors.
5. **Digital Signature Module** (as per objectives)
    - Ensures authenticity of logged events.

Outputs

Immutable block added to the ledger.

11.5 Module 5 — Dashboard, Analytics & Reporting

Purpose

To visualize transactions, fraud alerts, blockchain logs, and analytics.

Subcomponents

1. **Real-Time Monitoring UI**
    - Live transaction stream (color-coded).
2. **Fraud Alert Dashboard**
    - Detailed view of suspicious transactions.
3. **Blockchain Explorer**
    - Shows each block + chain link integrity.
4. **Analytics Engine**
    - Trends, heatmaps, time-series, risk matrices.
5. **Export Engine**
    - PDF/CSV export of:
        - alerts
        - blockchain logs
        - model performance

Outputs

Dynamic UI, download-ready reports, and interactive insights.

SECTION 12 — DATA FLOW ARCHITECTURE (END-TO-END PIPELINE)

This section describes the **complete data lifecycle**, from ingestion to audit verification. Designed using your Level-0 and Level-1 DFD diagrams within the uploaded report.

78_Sem-1_report

12.1 Data Flow Summary

Step 1 — Transaction Input

- System receives a transaction from CSV or simulated stream.
- Each record includes:
  `{TxnID, UserID, Amount, Time, Device, Location, Merchant}`

Step 2 — Preprocessing Pipeline

- Cleans + normalizes data.
- Generates derived features (spend patterns, location deviation, user baseline metrics).

Step 3 — AI Fraud Scoring

- ML model predicts fraud score (0-1).
- Threshold applied → classify as “Legit” or “Suspicious”.

Step 4 — Explainability Layer

- SHAP values generated.
- XAI narrative produced.

Step 5 — Blockchain Logging

For suspicious transactions:

- New block created.
- Previous hash attached.
- Block added to ProofLedger.

Step 6 — Dashboard Visualization

- Live feed updated instantly.
- Fraud alerts displayed in red.
- Blockchain explorer shows new block.

Step 7 — Reporting & Audits

- System generates PDF/CSV analytics.
- Auditors verify blockchain chain integrity.

12.2 Data Flow Diagram — Conceptual (Textual Form)

Input → Preprocessing → AI Engine → XAI → Blockchain → Dashboard → Reports

Or detailed version:

```
Transaction → Validate → Clean → Feature Engineering
           → Model Prediction → Fraud Score
           → XAI Module → Explanation
           → Blockchain Block Creation → Append Block
           → Dashboard UI → Analytics → Export
```

12.3 Interaction Flow with External Actors

Admin

- Configures thresholds
- Manages model updates
- Oversees blockchain integrity checks

Analyst

- Views real-time alerts
- Reviews XAI explanations
- Performs fraud investigations

Auditor

- Performs chain integrity validation
- Downloads reports
- Conducts compliance verification

SECTION 13 — DATASET SPECIFICATION (FULL ENTERPRISE SCHEMA)

This is one of the most critical sections.
It defines **exact dataset requirements**, grounded in fraud detection literature and your project files.

13.1 Dataset Format

- File Type: CSV or XLSX
- Encoding: UTF-8
- Minimum Rows: **5,000+**
- Maximum Rows: **300,000+**
- Use Case: Real-time classification + anomaly detection
- Clean, normalized formatting

13.2 Mandatory Columns (with Descriptions)

Column Name	Type	Description
`transaction_id`	String	Unique ID for each transaction
`user_id`	String/Int	Customer account/user identifier
`amount`	Float	Transaction amount in INR
`timestamp`	Datetime	Exact time of transaction
`location`	String	City/country of transaction
`device_id`	String	Unique device identifier
`merchant_type`	String	Merchant category (electronics, food, etc.)
`payment_method`	String	Card/UPI/Wallet/etc.
`is_new_location`	Boolean	Whether location deviates from user history
`is_new_device`	Boolean	First-time device usage
`avg_spend_user`	Float	User’s average historical spend
`deviation_amount`	Float	Distance from average spend
`label`	Int (0/1)	Fraud / Not Fraud (supervised)

13.3 Optional Fields

Field	Description
`ip_address`	Helps detect suspicious login patterns
`latitude_longitude`	Allows geo-mapping fraud heatmaps
`merchant_id`	Identifies merchant risk clusters
`day_of_week`	Captures weekly transaction habits

13.4 Dataset Schema Table

A complete schema example:

```
transaction_id (string, PK)
user_id (string)
timestamp (datetime)
amount (float)
merchant_type (string)
payment_method (string)
device_id (string)
location (string)
ip_address (string, optional)
avg_spend_user (float)
deviation_amount (float)
is_new_device (bool)
is_new_location (bool)
label (int: 0/1)
```

13.5 Dataset Rules

- No missing values for: `txn_id`, `amount`, `timestamp`.
- Numerical fields must be non-negative.
- Timestamps must be monotonic.
- UTF-8 encoding mandatory.

SECTION 14 — DATASET SOURCES (KAGGLE & OPEN DATA)

Below are **validated, domain-specific Kaggle datasets** suitable for fraud detection.
All datasets match your system's feature requirements.

14.1 Kaggle Datasets for Fraud Detection

1. Credit Card Fraud Detection Dataset

- URL: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud
- Contains anonymized transaction and fraud labels.

2. Synthetic Financial Datasets for Fraud Detection

- URL: https://www.kaggle.com/datasets/ealaxi/paysim1
- Mobile money transaction simulation (≈7M rows).

3. Bank Account Fraud Dataset (Synthetic)

- URL: https://www.kaggle.com/datasets/ealaxi/bank-account-fraud
- Realistic fraud scenarios with labels.

4. Online Payments Fraud Dataset

- URL: https://www.kaggle.com/datasets/jainilcoder/online-payments-fraud-detection
- Includes device, amount, type, fraud label.

5. UPI/Wallet-Type Synthetic Fraud Dataset

(Simulated datasets available on Kaggle under digital wallets / fintech categories.)

All datasets are **open-source**, compatible with BE-level implementation.

SECTION 15 — DATASET VALIDATION & PREPROCESSING RULES

This is a mandatory enterprise section in your Master Prompt.

15.1 Data Cleaning Rules

- Remove duplicates using `transaction_id`.
- Drop entries with null `amount`, `user_id`, `timestamp`.
- Verify timestamp consistency and convert to standard format.

15.2 Missing Value Handling

- Numerical fields: mean/median imputation.
- Categorical fields: most frequent category.
- Highly corrupted rows: remove.

15.3 Outlier Handling

- Use Z-score or IQR for amount anomalies.
- Flag extreme values for manual review.

15.4 Feature Transformation

- Log-transform high-variance fields (amount).
- Normalize numerical features.
- One-hot encode categorical features.

15.5 Label Processing

- Convert fraud labels into:
  - `0 = legitimate`
  - `1 = fraud`

15.6 Class Imbalance Handling

- Apply SMOTE/oversampling for minority fraud class.
- Use class weighting for ML models.

15.7 Text Processing Rules

(If merchant descriptions included)

- Lowercase
- Remove punctuation
- Tokenize if needed
- Apply TF-IDF vectorization (optional)

15.8 Final Validation Rules

- Ensure zero missing values in AI input.
- Confirm normalized scaling.
- Verify no unseen categories in test data.
- Validate fraud labels before training.

READY FOR NEXT BATCH

Reply **next** and I will generate:

SECTION 16 to SECTION 20

(Algorithm Workflow, Pseudocode, Training Strategy, Complexity Analysis, Limitations)

Sources

You said:

next

ChatGPT said:

Understood. Proceeding with the next batch.

Below are **SECTION 16 to SECTION 20**, fully enterprise-grade, highly technical, and aligned with all your uploaded files:

- **78_Sem-1_report.docx**

  78_Sem-1_report

- **78_Synopsis.docx**

  78_Synopsis

- **Custom blockchain.docx**

  custome blockchain

- **Detailed Project Objectives.docx**

  DETAILED PROJECT OBJECTIVES

- **Final Objectives PDF**

  KRIXION_TECH_Final_Objectives

- **Final Problem Statement PDF**

  KRIXION_TECH_Final_Problem_Stat…

SECTION 16 — ALGORITHM / MODEL WORKFLOW (END-TO-END)

This section explains the **complete technical pipeline**, from raw transaction ingestion to blockchain-based fraud evidence storage.

The workflow is built using enterprise ML patterns, XAI modules, and blockchain hashing principles from your uploaded blockchain spec.

custome blockchain

16.1 Stage 1 — Data Ingestion & Feature Engineering

1. **Input Stream Acquisition**
   - Live simulator / batch CSV ingestion.
   - Mandatory fields validated.
2. **Data Cleaning**
   - Missing values handled.
   - Outliers processed (Z-score / IQR).
   - Invalid timestamps rejected.
3. **Feature Engineering**
   - Encoding categorical variables.
   - Derived metrics:
     - `amount_deviation`

        - `location_difference_score`
        - `device_change_flag`
        - `spend_frequency_score`
    - Scaling using MinMax/StandardScaler.

16.2 Stage 2 — Machine Learning Fraud Detection

The ML engine computes a **fraud probability score** between **0 to 1**.

Model Options

- Logistic Regression
- Random Forest
- Isolation Forest (anomaly detection)
- Hybrid Ensemble (optional)

Decision Rule

```
If fraud_score ≥ threshold θ:
       label = "Suspicious"
Else:
       label = "Legitimate"
```

Threshold typically **θ = 0.60** (tuned using ROC/Precision-Recall tradeoff).

16.3 Stage 3 — Explainable AI (XAI) Engine

Integrated based on your objectives

KRIXION_TECH_Final_Objectives

.

Process

1. SHAP values computed for each feature.
2. Top-k contributing factors selected.
3. Human-readable explanation generated:
    - “High amount deviation + new device + foreign location.”

Outputs

- Feature contribution list
- Explanation narrative
- Confidence score

16.4 Stage 4 — Blockchain Logging (ProofLedger)

Aligned with your blockchain document

custome blockchain

.

Only **fraudulent or high-risk** transactions are written to the blockchain.

Steps:

1. Construct block:

```
{
  index,
  timestamp,
  transaction_data,
  fraud_score,
  explanation_hash,
  prev_block_hash,
  digital_signature
}
```

11. Generate SHA-256 hash.
12. Append block to the chain.
13. Validate chain integrity.

16.5 Stage 5 — Dashboard Visualization

Output components:

- Real-time transaction stream
- Fraud alerts
- XAI analysis panel
- Blockchain explorer
- Trend analytics

16.6 Stage 6 — Reporting & Compliance Outputs

- PDF summary
- CSV logs
- Blockchain verification report
- Model metrics report

SECTION 17 — ALGORITHM PSEUDOCODE (MANDATORY)

Required by your master prompt.
Below is complete enterprise-grade pseudocode for:

- Preprocessing
- TF-IDF (if text fields exist)
- Model Training
- Model Inference
- XAI
- Blockchain Logging

17.1 Data Preprocessing Pseudocode

```
FUNCTION preprocess_data(input_df):
    REMOVE duplicates FROM input_df
    HANDLE missing values:
        numeric → median
        categorical → mode
    FOR each column IN categorical_columns:
        input_df[column] = one_hot_encode(column)
    SCALE numerical_columns USING StandardScaler
    COMPUTE derived_features(input_df)
    RETURN input_df
```

17.2 Model Training Pseudocode

```
FUNCTION train_model(dataset):
    X, y = split_features_labels(dataset)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
    model = RandomForestClassifier()
    model.fit(X_train, y_train)
    SAVE model AS model.pkl
```

```
    RETURN model, accuracy_score(y_test, model.predict(X_test))
```

17.3 Model Inference / Real-Time Fraud Scoring

```
FUNCTION predict_transaction(txn_record):
    txn_processed = preprocess_single_record(txn_record)
    fraud_score = model.predict_proba(txn_processed)[1]
    IF fraud_score >= threshold:
        label = "Suspicious"
    ELSE:
        label = "Legitimate"
    RETURN fraud_score, label
```

17.4 XAI (SHAP) Pseudocode

```
FUNCTION generate_explanation(model, txn_record):
    explainer = shap.TreeExplainer(model)
    shap_values = explainer.shap_values(txn_record)
    top_features = SELECT top contributing features
    narrative = convert_to_human_readable(top_features)
    RETURN shap_values, narrative
```

17.5 Blockchain Block Creation Pseudocode

```
FUNCTION create_block(transaction, fraud_score, explanation):
    block = {
        "index": len(chain) + 1,
        "timestamp": current_time(),
        "txn": transaction,
        "score": fraud_score,
        "explanation_hash": sha256(explanation),
        "prev_hash": chain[-1].hash
    }
    block.hash = sha256(block)
    APPEND block TO chain
    RETURN block
```

17.6 Blockchain Validation

```
FUNCTION validate_chain(chain):
    FOR i FROM 1 TO len(chain) - 1:
        IF chain[i].prev_hash != chain[i-1].hash:
            RETURN False
        IF sha256(chain[i].data) != chain[i].hash:
            RETURN False
    RETURN True
```

SECTION 18 — MODEL TRAINING STRATEGY

Based on your system design and objectives, below is the formal strategy.

18.1 Data Split Strategy

- **Train–Test Split:** 80/20
- **Cross-Validation:** 5-fold
- **Validation Metrics:**
    - Accuracy
    - Precision
    - Recall
    - F1-score
    - ROC-AUC

18.2 Handling Imbalanced Datasets

- Apply **SMOTE** on training data.
- Adjust classifier weights:
- `class_weight = {0:1, 1:5}`

18.3 Feature Engineering Strategy

- Encode categorical fields.
- Apply log-scaling for skewed amounts.
- Generate user-behavior profiles (mean spend, last location, device history).

18.4 Hyperparameter Tuning

Tuned using GridSearchCV:

- Random Forest:
    - n_estimators: 100–500
    - max_depth: 5–20
    - min_samples_split: 2–10

- Logistic Regression:
  - C: 0.1–10
  - penalty: l2

18.5 Model Refresh Policy

- Scheduled retraining every **N days** (configurable).
- Retrain when:
  - False positives spike
  - Distribution drift detected
  - New dataset appended

SECTION 19 — COMPUTATIONAL COMPLEXITY ANALYSIS

19.1 Preprocessing Complexity

- One-hot encoding: **O(N × K)**
- Scaling: **O(N)**
- Derived features: **O(N)**
  Where:
  N = number of rows
  K = number of categorical categories

19.2 ML Training Complexity

Random Forest

- Complexity: **O(T × N × log N)**
  Where T = number of trees.

Logistic Regression

- Complexity: **O(N × D)**
  Where D = number of features.

19.3 Inference Complexity

- Random Forest inference: **O(T × D)** (fast)
- Logistic Regression inference: **O(D)** (very fast)

19.4 SHAP Computation Complexity

- Tree SHAP: **O(T × D)**
- Kernel SHAP (if used): **O(N²)** → expensive (avoid in real-time)

19.5 Blockchain Complexity

- Block creation: **O(1)**
- Hash computation: **O(K)** where K = block size
- Chain validation: **O(B)** where B = number of blocks

SECTION 20 — LIMITATIONS OF CURRENT ALGORITHM

20.1 ML Limitations

1. **Model is dependent on dataset quality**
   Poor or biased data → poor detection.
2. **Imbalanced fraud class issue**
   Even with SMOTE, some minority patterns remain underrepresented.
3. **Static thresholding**
   Optimal threshold varies across time and datasets.

4. **Cold-start problem**
   New users have no baseline behavior → harder detection.

20.2 Explainable AI Limitations

1. **SHAP is computationally expensive**
   Heavy computation for large transactions.
2. **Interpretation is approximate**
   SHAP approximations may not reflect deep model logic perfectly.

20.3 Blockchain Limitations

1. **Local blockchain is not distributed**
   It is a simulation, not a multi-node network.
   (As designed intentionally for offline BE project.)

   custome blockchain

2. **Ledger can grow large over time**
   Requires pruning or archival strategies.
3. **Does not support smart contracts**
   Only immutable logging is implemented.

20.4 System Limitations

- Runs on a single machine: no horizontal scaling.
- Real-time model retraining not automated.
- No integration with external APIs (as per offline requirement).

NEXT BATCH READY

Reply **next** and I will deliver:

SECTION 21 to SECTION 25

(Future Enhancements, Analytical Outputs, Visualizations, Mathematical Results, Business Insights)

Sources

You said:

next

ChatGPT said:

Understood. Proceeding with the next batch.

Below are **SECTION 21 to SECTION 25**, all enterprise-grade, aligned with your uploaded files:

- **78_Sem-1_report.docx**

  78_Sem-1_report

- **78_Synopsis.docx**

  78_Synopsis

- **Custom blockchain.docx**

  custome blockchain

- **Detailed Objectives.docx**

  DETAILED PROJECT OBJECTIVES

- **Final Objectives (KRIXION TECH).pdf**

  KRIXION_TECH_Final_Objectives

- **Final Problem Statement.pdf**

  KRIXION_TECH_Final_Problem_Stat…

SECTION 21 — FUTURE ALGORITHM ENHANCEMENTS

This section outlines enterprise-class next-step upgrades that can be integrated into future versions of the system.

21.1 Integration of Transformer-Based Models (BERT / RoBERTa)

If merchant descriptions, complaint messages, or narrative fields are included, transformer-based models significantly enhance fraud text analysis:

- BERT for semantic anomaly detection
- RoBERTa for merchant-risk scoring
- DistilBERT for on-device lightweight inference

Benefits:

- Increased fraud detection accuracy
- High contextual understanding
- Excellent for multi-lingual FinTech scenarios

21.2 Deep Learning (LSTM / GRU) for Sequence Pattern Fraud

Fraud is often temporal (rapid repeated transactions, abnormal time/behavior drift).

Enhancement:

- LSTM or GRU model to encode user behavior sequences:
    - time gaps between transactions
    - session-based patterns
    - transaction velocity

Benefits:

- Improved detection for “burst” fraud
- Strong for behavioral anomaly detection

21.3 Topic Modeling (LDA or BERTopic)

Useful when merchant descriptions or fraud complaint narratives exist.

- Auto-cluster risk factors
- Identify new fraud categories emerging over time
- Feed themes into fraud alerting logic

21.4 Online Learning / Incremental Models

Implement algorithms that update fraud logic continuously:

- Incremental SGD Classifier
- River library models
- Online Isolation Forest

Benefits:

- Model adapts to new fraud patterns without full retraining
- Real-time fraud intelligence

21.5 Multi-Language Support

For geographic expansion:

- Integrate multilingual transformer models
- Support Indian languages, English, and others
- Local merchant descriptions become analyzable

21.6 Cloud-Inference Ready Architecture (Optional)

Even though your system is fully offline now, future enhancements may include:

- Optional deployment to AWS / GCP
- Auto-scaling microservices
- Distributed blockchain nodes

Not required for BE-level offline project, but useful for enterprise scaling.

SECTION 22 — EXPECTED ANALYTICAL OUTPUTS

These outputs appear in the dashboard and are required according to your objectives and the academic report.

78_Sem-1_report

22.1 Sentiment / Risk Scoring Summary

- List of total transactions
- Number of legitimate vs suspicious ones
- Average fraud risk score
- Highest-risk user accounts
- Daily/weekly fraud trend changes

22.2 Theme Extraction Summary

Using SHAP + categorical patterns:

- Top fraud-causing parameters (amount deviation, new location)
- Merchant categories with highest fraud rate
- Device types most associated with risk
- Geographical clusters

22.3 Severity Scoring Summary

- Severity Level 1: Low risk
- Severity Level 2: Medium risk
- Severity Level 3: High risk
- Severity Level 4: Critical (immediate block recommended)

Severity rules are derived from XAI explanations + fraud score thresholds.

22.4 Blockchain Integrity Reports

- Number of blocks created
- Validation status (Valid/Tampered)
- Timestamp distribution
- Chain growth metrics

22.5 Model Performance Summary

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC
- Confusion matrix summary

SECTION 23 — VISUALIZATION OUTPUTS (Required as per MASTER PROMPT)

Your system MUST generate the following downloadable charts (PNG format):

23.1 Fraud Sentiment Bar Chart

- Legitimate vs Fraudulent transactions
- Based on classification labels

23.2 Fraud Distribution Pie Chart

- Percentage distribution
- Useful for high-level executive reporting

23.3 Time-Series Line Chart

- Fraud trend across hours, days, or weeks
- Derived from timestamp analysis

23.4 Heatmap (Geographical or Feature Correlation)

- If geo-data available → location heatmap
- Else → correlation matrix heatmap (amount, device, merchant)

23.5 Theme Frequency Bar Chart

- Using XAI explanation tokens
- Shows which factors caused the most fraud alerts

23.6 Word Cloud (Optional)

- Merchant names
- Description fields
- Fraud patterns

23.7 ROC Curve

- Full classifier performance visualization

23.8 Confusion Matrix Heatmap

- True/False positive patterns
- Required for academic evaluation

23.9 Clustering Scatter Plot

- Apply KMeans or DBSCAN
- Shows natural clusters of fraudulent patterns

23.10 t-SNE / PCA Projection

- Dimensionality reduction for visualizing fraud vs normal clusters
- Mandatory for an enterprise ML report

SECTION 24 — STATISTICAL & MATHEMATICAL RESULTS

These results MUST appear in the report and dashboard exports.

24.1 TF-IDF Vocabulary Size (if text present)

- Vocabulary size = total unique merchant terms
- Average TF-IDF weight distribution

24.2 Dataset Sparsity Report

If encoded categorical variables create sparse matrices:

- Sparsity = (# zero elements / total elements)
- Helps in optimization

24.3 Average Review / Transaction Length

- Mean number of attributes per record
- Mean time gap between transactions

24.4 Variance & Standard Deviation

For numerical attributes:

- amount
- user_avg_spend
- transaction_frequency

24.5 Fraud Score Distribution

- Histogram of ML scores
- Threshold selection analysis

24.6 Correlation Plots

For numerical attributes:

- amount
- location risk score
- device frequency

24.7 Regression Plots

If additional regression metrics are studied:

- Fraud risk vs amount
- Fraud risk vs spend deviation

Even though the core model is classification-based, regression plots give deeper financial insights.

SECTION 25 — BUSINESS INSIGHTS (PREMIUM OUTPUT)

This section provides executive-level findings expected by managers, auditors, and enterprise clients.
These insights are directly used in academic viva and also in fintech industry presentations.

25.1 Top 10 Fraud Issues Identified

Examples:

1. Sudden high-value transactions
2. Transactions from new device-location combinations
3. Rapid multiple transactions within a short timeframe
4. Transactions during abnormal time windows
5. Out-of-pattern spending compared to user baseline
6. Repeated failed attempts before success
7. High-risk merchant categories
8. Location anomalies (international/long distance)
9. Suspicious device fingerprint behavior
10. Low-velocity account with sudden high-value activity

25.2 Top 10 Positive Insights

- Majority of transactions follow stable user patterns
- Typical users maintain consistent locations
- Low deviation in device fingerprints
- Genuine users rarely transact at unusual times

25.3 Cross-App Comparison (if multiple datasets)

- UPI fraud vs Credit-card fraud differences
- Wallet transactions show higher anomaly frequency
- Card-not-present transactions show specific risk clusters

25.4 Version Comparison (Optional)

Comparing ML models:

- v1: Logistic Regression
- v2: Random Forest
- v3: Ensemble

Enterprise deliverables must show improvements across versions.

25.5 Priority Matrix

A 2x2 risk matrix:

Impact	Low	High
Low Probability	Monitor	Review
High Probability	Flag	Block Immediately

Used by fraud analysts for prioritization.

25.6 Developer-Focused Recommendations

1. Add model drift detector
2. Add incremental learning (online ML)
3. Add multi-node blockchain simulation (optional)
4. Build more aggressive feature engineering
5. Implement anomaly-based velocity checks
6. Add clustering-based unsupervised fraud detection

NEXT BATCH READY

Reply **next** and I will deliver:

SECTION 26 to SECTION 30

(Result Export Requirements, Testing Strategy, Model Validation, Acceptance Testing, API Contract)

Sources

You said:

next

ChatGPT said:

Understood. Proceeding with the next batch.

Below are **SECTION 26 to SECTION 30**, fully enterprise-grade, aligned with your uploaded files:

- **78_Sem-1_report.docx**

  78_Sem-1_report

- **78_Synopsis.docx**

  78_Synopsis

- **Custome blockchain.docx**

  custome blockchain

- **Detailed Project Objectives.docx**

  DETAILED PROJECT OBJECTIVES

- **Final Objectives (KRIXION).pdf**

  KRIXION_TECH_Final_Objectives

- **Final Problem Statement.pdf**

  KRIXION_TECH_Final_Problem_Stat…

SECTION 26 — RESULT EXPORT REQUIREMENTS

All final outputs generated by the system must be exportable in secure, lightweight, offline-compatible formats.

These requirements are **non-negotiable enterprise deliverables**.

26.1 Chart Export Requirements (PNG)

System must export high-quality PNG images for:

1. Fraud sentiment bar chart
2. Fraud distribution pie chart
3. Time-series trend line chart
4. Correlation heatmap
5. Theme-frequency bar chart
6. Word cloud (optional)
7. ROC curve
8. Confusion matrix heatmap
9. Fraud clustering scatter plot
10. PCA/t-SNE visualization

All PNGs must be:

- Minimum resolution: **1280 × 720**
- Stored under `/exports/charts/`

26.2 Table Export Requirements (CSV)

Exports required:

- Processed dataset (post-cleaning & post-preprocessing)
- Fraud alert table
- Suspicious transaction logs
- Block-by-block blockchain ledger
- XAI feature-importance tables
- Model performance metrics

All CSVs stored under:

`/exports/csv/`

26.3 PDF Report Requirements

System must auto-generate a PDF report containing:

A. Executive Summary

- Fraud detection performance
- Blockchain integrity status

B. Analytical Insights

- Top 10 fraud-causing reasons
- Trend summaries
- Device/location analysis

C. Model Metrics

- Accuracy, Precision, Recall, F1
- ROC Curve screenshot
- Confusion matrix

D. Blockchain Report

- Number of blocks
- Validation result
- Tamper verification

E. Recommendations

- Analyst actions
- Developer actions

PDF saved under:

```
/exports/pdf/Final_Fraud_Report.pdf
```

26.4 Recommendation Sheet

A separate CSV or PDF providing:

- Recommended actions per fraud case
- Severity category
- Suggested investigation steps
- Notes for auditors

26.5 Internal System Logs (Optional Export)

- Error logs
- Model inference logs
- Blockchain append logs

Stored under:

`/exports/logs/`

SECTION 27 — TESTING STRATEGY

This section defines the **complete QA and ML testing framework.**

27.1 Unit Testing

Unit tests must cover:

- Data ingestion functions
- Preprocessing functions
- Feature engineering
- Scaling & encoding
- Model loader & inference engine
- SHAP explanation generator
- Blockchain block creation
- Hash validation
- Digital signature verification

Tools:
`pytest`, `unittest`, or `nose2`

27.2 Integration Testing

Testing the interaction between modules:

- Preprocessing → ML model
- ML model → XAI engine
- XAI engine → Blockchain
- Blockchain → Dashboard
- End-to-end API pipeline

Focus:

- Data integrity across modules
- Performance under load

27.3 System Testing

Full system executed on local machine:

- Live transaction simulation
- Immediate prediction
- Real-time dashboard update
- Block creation for suspicious cases
- Chain validation routine

System must pass:

- Stability test for 10,000+ transactions
- Zero crashes under continuous load

27.4 Regression Testing

Whenever:

- Model retrained
- Threshold changed
- Block structure updated
- Dashboard modified

System must:

- Maintain backward compatibility
- Preserve blockchain integrity
- Preserve model performance benchmark

27.5 UI Testing

Using **Selenium / Playwright** (optional for BE project):

- Button interactions
- Table rendering
- Chart rendering
- Navigation flow
- Role-based visibility (RBAC enforcement)

27.6 Error-Handling Testing

Test scenarios:

- Missing fields
- Invalid timestamps
- Device anomalies
- Block tampering
- ML inference error
- SHAP computation failure

System should fail safely with descriptive error messages.

SECTION 28 — MODEL VALIDATION TESTING

This section defines how the ML model is validated before deployment.

28.1 Accuracy Score

Target: **≥ 95%** (based on your objectives)

DETAILED PROJECT OBJECTIVES

Formula:

```
accuracy = correct_predictions / total_predictions
```

28.2 Precision

Measures correctness of flagged fraud cases.

```
precision = TP / (TP + FP)
```

Goal: **High precision** to reduce false alarms.

28.3 Recall (Sensitivity)

Measures how many fraud cases are correctly detected.

```
recall = TP / (TP + FN)
```

Goal: **High recall** for fraud detection.

28.4 F1-Score

Balance of precision and recall.

Required: **F1 ≥ 0.90**

28.5 ROC-AUC

Area under curve must be:

- **≥ 0.95** for enterprise acceptance
- Shows model robustness

28.6 Cross-Validation (5-Fold)

- Model must generalize well
- Standard deviation across folds < **0.03**

28.7 Overfitting/Underfitting Check

Criteria:

- Training accuracy should not exceed test accuracy by >10%
- Learning curves must converge
- SHAP global explanation patterns must be stable

SECTION 29 — ACCEPTANCE TESTING

Defines measurable acceptance criteria for final submission.

29.1 Functional Acceptance Criteria

Requirement	Acceptance Criteria
Fraud detection	≥95% accuracy
Real-time scoring	<150 ms latency
Blockchain logging	Guaranteed immutability
Dashboard	Updates within 1 sec
XAI	Explanation must be generated per suspicious txn
Role-based access	RBAC enforced 100% offline

29.2 Performance Acceptance Criteria

- Must process **≥100 transactions/second** in batch mode
- Must handle a dataset of **100k rows** without crash

29.3 Security Acceptance Criteria

- No raw sensitive data in logs
- Blockchain must detect any tampering
- SQLite database protected with hashed sensitive fields

29.4 Offline Acceptance Criteria

- Entire system must run **without internet**
- All dependencies must be installed locally
- Reports, blockchain, and AI must work offline

29.5 Usability Acceptance Criteria

- UI must load in <2 seconds
- Navigation must require max 2 clicks to reach core features
- Exported files must be readable on evaluator devices

29.6 Final Acceptance Checklist

- All charts generated
- All reports exported
- Blockchain validation passed
- ML validation passed
- No crash in continuous run for 5 minutes

SECTION 30 — API CONTRACT

Even though your system is offline and dashboard-based, internal APIs are used between modules.

Below is the enterprise API specification.

30.1 API 1 — `/predict`

Method: POST
Description: Returns fraud score + label

Input JSON:

```
{
  "transaction_id": "...",
  "user_id": "...",
  "amount": 2000.5,
  "timestamp": "2025-01-10 12:30:00",
```

```
  "device_id": "...",
  "location": "...",
  "merchant_type": "...",
  "payment_method": "..."
}
```

Output JSON:

```
{
  "fraud_score": 0.83,
  "label": "Suspicious",
  "explanation": "High deviation + new device"
}
```

30.2 API 2 — `/explain`

Method: POST
Purpose: Generate SHAP-based explanation

Input: Preprocessed transaction

Output:

```
{
  "shap_values": [...],
  "top_features": ["amount_deviation", "new_device"],
  "narrative": "Transaction flagged due to..."
}
```

30.3 API 3 — `/add_block`

Method: POST
Purpose: Add suspicious transaction to blockchain

Output:

```
{
 "block_index": 101,
 "status": "added",
 "chain_valid": true
}
```

30.4 API 4 — `/validate_chain`

Method: GET
Output:

```
{
 "valid": true,
 "total_blocks": 150
}
```

30.5 API 5 — `/analytics`

Method: GET
Generates dashboard charts & statistics.

NEXT BATCH READY

Reply **next** and I will deliver:

SECTION 31 to SECTION 35

(UI/UX Specification, Database Schema, Security Considerations, Deployment Plan, Version Management)

Sources

You said:

next

ChatGPT said:

Understood. Proceeding with the next batch.

Below are **SECTION 31 to SECTION 35**, fully enterprise-grade and aligned with all uploaded files:

- **78_Sem-1_report.docx**

  78_Sem-1_report

- **78_Synopsis.docx**

  78_Synopsis

- **Custom blockchain.docx**

custome blockchain

- **Detailed Objectives.docx**

  DETAILED PROJECT OBJECTIVES

- **Final Objectives (KRIXION TECH).pdf**

  KRIXION_TECH_Final_Objectives

- **Final Problem Statement.pdf**

  KRIXION_TECH_Final_Problem_Stat…

SECTION 31 — UI/UX SPECIFICATION

This section defines the complete interface design for the real-time AI + Blockchain Fraud Detection platform.
The UI must be **minimal, fast, offline-functional, and role-based**.

31.1 UI Design Principles

- **Lightweight** (must run smoothly on 4–8 GB RAM laptops).
- **Real-time visualization** (auto-refresh without page reload).
- **Role-based visibility** (Admin, Analyst, Auditor).
- **Error-free navigation** (max 2 clicks to reach major sections).
- **Mobile-responsive** (optional).
- **Minimalist enterprise color scheme** (Blue/Grey/White).

31.2 Core Screens

1. Login / Role Authentication

- Username
- Password
- Role selection (Admin / Analyst / Auditor)
- Local authentication (SQLite-based)

---

2. Home Dashboard (Real-Time Monitoring Panel)

Contains:

- Live transaction table (auto-scroll)
- Status color codes:
    - Green = Legit
    - Red = Suspicious
    - Yellow = Monitor
- Fraud score badge
- Timestamp and location indicators

Widgets:

- "Total Transactions Today"
- "Suspicious Transactions Count"
- "Current Model Accuracy"
- "Blockchain Validity Status"

---

3. Fraud Alerts Panel

Detailed insights for flagged transactions:

- Transaction ID
- User ID
- Device ID
- Risk Score
- XAI explanation ("Why flagged?")
- Severity Level (1–4)

Filters:

- Severity
- Location
- Merchant Type
- Device Type

---

4. XAI Interpretation View

Shows:

- SHAP value bar chart
- Top 5 contributing features
- Human-readable explanation
- Confidence score

---

5. Blockchain Explorer (ProofLedger Viewer)

Displays:

- Block index
- Timestamp
- Transaction data summary
- Previous block hash
- Current hash
- Digital signature
- Chain validity indicator

Ability to:

- Expand block
- Download block details
- Run chain validation

---

6. Analytics & Reports Page

Charts:

- Fraud trends
- Heatmaps
- Merchant risk plots
- ROC curve
- Confusion matrix

Buttons:

- Export PDF
- Export CSV
- Export PNG charts

---

7. Admin Panel

Admin-exclusive features:

- Threshold configuration
- Model retraining
- Model version control
- Blockchain reset (controlled)
- User-role management

SECTION 32 — DATABASE SCHEMA

SQLite-based design for complete offline support.

32.1 Database Overview

Three primary database groups:

1. **Transaction Database**
2. **Blockchain Ledger Database**
3. **User Access & Configuration Database**

32.2 Table Definitions

---

1. `transactions` Table

Column	Type	Description
txn_id	TEXT (PK)	Unique transaction ID
user_id	TEXT	ID of customer
amount	REAL	Transaction amount
timestamp	TEXT	Time of transaction
location	TEXT	City or Region
device_id	TEXT	Device fingerprint
merchant_type	TEXT	Merchant category
payment_method	TEXT	Card/UPI/Wallet
fraud_score	REAL	ML score

Column	Type	Description
label	TEXT	Legit / Suspicious
shap_summary	TEXT	Top contributing features
severity	INTEGER	Level 1–4

---

2. `blockchain` Table

Column	Type	Description
block_index	INTEGER (PK)	Index in chain
timestamp	TEXT	Block creation time
txn_data	TEXT	JSON of transaction details
fraud_score	REAL	Score logged
explanation_hash	TEXT	SHA-256 of explanation
prev_hash	TEXT	Hash of previous block
current_hash	TEXT	SHA-256 block hash
signature	TEXT	Digital signature

Matches your blockchain doc.

custome blockchain

---

3. `users` Table

Column	Type	Description
username	TEXT (PK)	
password_hash	TEXT	
role	TEXT	Admin/Analyst/Auditor

---

4. `config` Table

Stores system-wide configs:

- Threshold
- Model version
- Blockchain status
- Dashboard refresh interval

32.3 Relationships

- `transactions` links logically to blockchain blocks (1 block per suspicious txn).
- `users` references RBAC.
- `config` controls global behavior.

No foreign keys required to keep system lightweight.

SECTION 33 — SECURITY CONSIDERATIONS

FinTech-grade security applied, even though system is offline.

33.1 Data Security

- Sensitive fields hashed (`device_id`, `user_id`).
- Encryption recommended for `transactions.db` (SQLite AES-based wrapper).
- No plain text logs containing PII.

33.2 Blockchain Integrity

- SHA-256 applied to all blocks.
- Previous-hash chain ensures immutability.
- Tamper detection must run daily or on admin command.

Matches your blockchain specification.

custome blockchain

33.3 Malware & Tampering Protection

- Disable external scripts.
- Validate all CSV uploads.
- Enforce strict exception handling.

33.4 Role-Based Access Control (RBAC)

As per objectives: Admin / Analyst / Auditor.

DETAILED PROJECT OBJECTIVES

Rules:

- Admin: Full access
- Analyst: View + Investigate
- Auditor: View-only + Blockchain verification

33.5 API Security

Even with local APIs:

- Input sanitization for POST requests
- Size limits enforced
- JSON schema validation used
- Local-only endpoints (no public exposure)

33.6 Model Security

- Prevent model poisoning by validating new training datasets.
- Keep `.pkl` files read-only.
- Ensure SHAP and model parameters are not user-modifiable.

SECTION 34 — DEPLOYMENT PLAN

System must be deployed **offline**, on a **single machine**, with zero cloud dependencies.

34.1 Deployment Steps

1. **Install Python 3.10+**
2. **Install required libraries:**
3. `pip install -r requirements.txt`
4. **Initialize SQLite database:**
    - Run `setup_database.py`
5. **Load initial ML model (`model.pkl`)**
6. **Start backend:**
7. `python app.py`
8. **Open dashboard:**
    - Localhost link: `http://127.0.0.1:8000`

34.2 Offline Deployment Requirements

- No need for: cloud database, internet, external servers
- All components bundled locally
- Blockchain stored locally as JSON + SQLite

34.3 Cross-Platform Packaging

Packaging for:

- Windows executable via PyInstaller
- Linux AppImage
- macOS .app bundle

This ensures evaluators can run it without Python.

34.4 Sample Deployment Folder Structure

```
/app
  /ai_engine
  /blockchain
  /dashboard
  /database
  /exports
  /logs
```

```
  config.json
  model.pkl
  app.py
```

34.5 Deployment Validation

System must pass:

- Cold start test
- Bulk upload test
- Blockchain validation test
- Real-time simulation test

SECTION 35 — VERSION MANAGEMENT

Enterprise-level version control architecture.

35.1 Version Naming Convention

```
vMajor.Minor.Patch
Example:
v1.0.0 (Initial BE submission)
v1.1.0 (Added XAI enhancements)
v2.0.0 (Added LSTM + Transformer support)
```

35.2 Folder Structure Versioning

Every release must include:

```
/releases
  /v1.0.0
  /v1.1.0
  /v2.0.0
```

35.3 Git-Based Version Control

Branch structure:

- `main` — latest stable release
- `dev` — active development
- `feature/*` — new modules (XAI, blockchain, analytics)
- `hotfix/*` — quick fixes

35.4 Model Versioning

ML models stored as:

```
/models
  model_v1.pkl
  model_v2.pkl
  model_v3.pkl
```

Metadata stored in `config` table:

- Version ID
- Training date
- Accuracy

35.5 Blockchain Version History

Chain snapshots stored in:

```
/blockchain/history/
```

Before major updates:

- Export full chain
- Validate integrity
- Save snapshot

35.6 Release Notes Format

Each release must include:

- Added
- Removed
- Updated
- Fixed
- Known Issues

NEXT BATCH READY

Reply **next** and I will generate:

SECTION 36 to SECTION 38 — FINAL PART

- 7-Day Execution Plan (Single Intern)
- Final Deliverables Checklist
- Final Submission Folder Structure

This will complete your full **38-Section Enterprise Specification Document**.

Sources

You said:

next

ChatGPT said:

Understood. Proceeding with the **final batch**.

Below are **SECTION 36 to SECTION 38**, completing the full 38-section Enterprise Project Specification Document.

All content is fully aligned with your uploaded files:

- **78_Sem-1_report.docx**

  78_Sem-1_report

- **78_Synopsis.docx**

  78_Synopsis

- **Custom blockchain.docx**

  custome blockchain

- **Detailed Objectives.docx**

  DETAILED PROJECT OBJECTIVES

- **KRIXION Final Objectives.pdf**

  KRIXION_TECH_Final_Objectives

- **Final Problem Statement.pdf**

  KRIXION_TECH_Final_Problem_Stat…

SECTION 36 — 7-DAY EXECUTION PLAN (SINGLE INTERN)

A complete VIBE-style rapid development plan enabling **one intern** to build and deliver the full project in **7 days**, with zero ambiguity.

This is one of the most important and enterprise-critical sections.

Day 1 — Environment Setup + Dataset Pipeline

Objectives:

- Prepare complete offline environment
- Build dataset ingestion and preprocessing pipeline

Tasks:

1. Install Python, dependencies, SQLite
2. Create folder structure
3. Build data loader functions
4. Implement data validation rules

5. Implement preprocessing steps (cleaning, scaling, encoding)
6. Implement feature engineering (amount deviation, device history, etc.)
7. Test preprocessing with sample dataset

Deliverables:

- `/app/data_pipeline/` fully ready
- Cleaned sample dataset
- Preprocessing logs

Day 2 — ML Model Development

Objectives:

- Build ML model for fraud detection
- Save model for inference

Tasks:

1. Train baseline model (Logistic Regression / Random Forest)
2. Implement train-test split + metrics evaluation
3. Hyperparameter tuning
4. Export `model.pkl`
5. Implement inference engine with thresholding logic
6. Unit test inference with 10+ synthetic transactions

Deliverables:

- `/app/ai_engine/model.pkl`
- `/app/ai_engine/inference.py`
- Accuracy ≥ 95% (as per objectives)

Day 3 — XAI Integration (SHAP)

Objectives:

- Generate explainability for each suspicious transaction

Tasks:

1. Integrate SHAP TreeExplainer
2. Generate per-transaction SHAP values
3. Convert SHAP results to human-readable narrative
4. Build XAI module
5. Test with 10–20 flagged transactions

Deliverables:

- `/app/xai_engine/explain.py`
- SHAP explanation outputs
- Top features extraction logic

Day 4 — Blockchain Development (ProofLedger)

Objectives:

- Implement custom blockchain simulation
- Ensure immutability using SHA-256

Tasks:

1. Implement block structure
2. Add SHA-256 hashing
3. Create linking chain (prev_hash)
4. Add digital signature feature
5. Build chain validation function
6. Store blocks in SQLite or JSON
7. Test chain tampering detection

Deliverables:

- `/app/blockchain/block.py`
- `/app/blockchain/chain.py`
- Valid/tampered chain tests
- Blockchain contains minimum 10 blocks

Day 5 — Real-Time Dashboard Development

Objectives:

- Build UI for monitoring + alerts + blockchain explorer + analytics

Tasks:

1. Build dashboard home screen
2. Build real-time transaction stream
3. Build fraud alerts panel
4. Build XAI viewer
5. Build blockchain explorer UI
6. Build analytics & charts page
7. Integrate backend APIs

Deliverables:

- Fully working dashboard
- All required charts render correctly
- Live auto-refresh

Day 6 — Reporting + Export Engine

Objectives:

- Implement PDF + CSV + PNG export features

Tasks:

1. Export alerts table (CSV)
2. Export blockchain ledger (CSV)
3. Export all charts (PNG)
4. Auto-generate PDF report with sections:
    - Model performance
    - Business insights
    - Fraud trends
    - Blockchain integrity

Deliverables:

- `/exports/pdf/`
- `/exports/csv/`
- `/exports/charts/`

Day 7 — Testing + Documentation + Final Integration

Objectives:

- Validate entire system
- Fix bugs
- Finalize academic + technical documentation

Tasks:

1. Run unit tests
2. Validate chain integrity
3. Validate SHAP explanations
4. Validate model performance
5. Validate offline execution
6. Create README + deployment guide
7. Prepare BE final-year academic documentation

Deliverables:

- Fully functional offline system
- Academic reports ready
- Deployment instructions
- Demo-ready build

SECTION 37 — FINAL DELIVERABLES CHECKLIST

A complete list of all items that must be submitted in your final project package.

37.1 Source Code Deliverables

- Complete codebase with modules:
    - `/ai_engine/`

    - `/xai_engine/`
    - `/blockchain/`
    - `/dashboard/`
    - `/data_pipeline/`
- `app.py` (main server)
- `config.json`
- `requirements.txt`

37.2 Dataset Files

- Raw dataset
- Cleaned dataset
- Feature-engineered dataset

37.3 Model Deliverables

- `model.pkl`
- Model training notebook (optional)
- Model performance metrics (CSV)
- SHAP global summary plot

37.4 Blockchain Deliverables

- Blockchain ledger file (JSON/SQLite)
- Blockchain integrity validation report
- Screenshots of chain explorer

37.5 Visualization Deliverables

PNG files for:

- Bar chart
- Pie chart
- Trend line
- Correlation heatmap
- Theme frequency

- Word cloud
- ROC curve
- Confusion matrix
- Clustering scatter plot
- PCA/t-SNE

37.6 Reporting Deliverables

- **Final PDF Report**:
  - Executive summary
  - ML results
  - Blockchain verification
  - Business insights
  - Visualizations
  - Conclusions
- **Technical Documentation**:
  - System architecture
  - Module architecture
  - Data flow diagrams
  - Blockchain structure
  - API documentation
- **User Manual (Evaluator Guide)**
- **Installation Guide**
- **Deployment guide**

37.7 BE Academic Deliverables

- BE final-year report (Synopsis + Sem-1 + Final)
- PPT for Viva
- 10–12 page research paper (optional)
- Plagiarism report (<5% expected)
- Working demonstration video

SECTION 38 — FINAL SUBMISSION FOLDER STRUCTURE

This is the **official, enterprise-standard directory layout** for final submission.

All evaluators expect this format.

```
Project_Fraud_Detection_ProofLedger/
│
├── app/
│   ├── ai_engine/
│   │   ├── model.pkl
│   │   ├── inference.py
│   │   └── train.py
│   │
│   ├── xai_engine/
│   │   ├── explain.py
│   │   └── shap_utils.py
│   │
│   ├── blockchain/
│   │   ├── block.py
│   │   ├── chain.py
│   │   ├── signature.py
│   │   └── ledger.json (or ledger.db)
│   │
│   ├── data_pipeline/
│   │   ├── preprocess.py
│   │   ├── feature_engineering.py
│   │   └── validation.py
│   │
│   ├── dashboard/
│   │   ├── templates/
│   │   ├── static/
│   │   ├── components/
│   │   └── dashboard.py
│   │
│   ├── exports/
│   │   ├── charts/
│   │   ├── csv/
│   │   └── pdf/
│   │
│   ├── logs/
│   ├── config.json
│   └── app.py
│
├── database/
│   └── transactions.db
│
├── documentation/
│   ├── Final_Report.pdf
│   ├── System_Architecture.pdf
│   ├── User_Manual.pdf
│   ├── Deployment_Guide.pdf
│   └── PPT/
│
├── datasets/
│   ├── raw/
│   ├── cleaned/
│   └── engineered/
│
```

```
├── models/
│   ├── model_v1.pkl
│   └── metrics.csv
│
├── releases/
│   ├── v1.0.0/
│   ├── v1.1.0/
│   └── v2.0.0/
│
├── requirements.txt
└── README.md
```

YOUR 38-SECTION ENTERPRISE PROJECT SPECIFICATION IS NOW COMPLETE